

person_tracker.py 파일 정리

전체 코드

```
# person_tracker.py
from ultralytics import YOLO

class PersonTracker:
    def __init__(
        self,
        model_path="yolo26s.pt",
        target_class="person",
        conf_threshold=0.5,
        tracker_config="bytetrack.yaml",
    ):
        self.model = YOLO(model_path)
        self.target_class = target_class
        self.conf_threshold = conf_threshold
        self.tracker_config = tracker_config

    def track(self, frame):
        results = self.model.track(
            frame,
            persist=True,
            tracker=self.tracker_config,
            verbose=False
        )

        tracked_persons = []

        if results is None or len(results) == 0:
            return tracked_persons

        result = results[0]

        if result.bboxes is None:
            return tracked_persons

        for box in result.bboxes:
            cls_id = int(box.cls[0])
            conf = float(box.conf[0])
            class_name = self.model.names[cls_id]

            if class_name != self.target_class:
```

```

        continue

    if conf < self.conf_threshold:
        continue

    # track_id가 없는 경우 방어 처리
    if box.id is None:
        continue

    person_id = int(box.id[0])

    x1, y1, x2, y2 = map(int, box.xyxy[0])

    tracked_persons.append({
        "person_id": person_id,
        "bbox": (x1, y1, x2, y2),
        "conf": conf,
        "class_name": class_name
    })

    return tracked_persons

```

클래스 역할 : 프레임 한 장을 입력받아서 사람만 탐지하고, 각 사람에게 추적 ID를 붙여 반환하는 클래스

- 사람 발견
- 사람마다 각각의 ID부여
- 다음 프레임에서도 같은 ID유지
- YOLO를 사용해서 객체 탐지, 객체 추적 가능

init()역할

- 모델 파일 로드
- 객체 추적 대상 선정
- 객체 신뢰도 기준 선정
- 객체 추적 알고리즘 선정 - 프레임마다 같은 객체 ID유지 알고리즘
 - 여기선 ByteTrack

track()역할

- 프레임 하나 입력받고 다음과 같은 리스트 형태로 반환 - 여러명일 수 있으니 리스트 사용

```

[
    {

```

```

        "person_id": 1,
        "bbox": (x1, y1, x2, y2),
        "conf": 0.87,
        "class_name": "person"
    }
]

```

- `persist=True` - 이전 프레임 기억 유지(없으면 프레임마다 새ID로 바뀜)
 - 이때 사용되는 알고리즘이 위에 ByteTrack
- 기타 `verbose`는 프레임마다 로그 콘솔에 도배되는거 방지해줌
- 예외처리
 - `if results is None or len(results) == 0:` 로 YOLO자체가 결과 생성 실패 판단
 - `if result.bboxes is None:` 로 YOLO는 동작했지만 카메라 화면에 아무도 없는지 판단
 - 둘 다 빈 리스트 반환
- 탐지된 객체 하나씩 검사하면서 클래스 번호를 이름으로 변환하고, 사람 아닌건 무시
 - YOLO는 일단 전부 검출 후 우리가 직접 필터링 걸어야함.
 - 신뢰도 낮은거 무시 - 현재 0.5 기준
 - 추적ID없으면 무시 - 가끔 추적 안될때 뺄나는거 방지
 - 사람 주변 좌표로 바운딩박스 좌표 만들어서 결과 리턴

외부 파일에서 사용하는 법

```

from person_tracker import PersonTracker

tracker = PersonTracker(model_path="yolo26s.pt")

persons = tracker.track(frame)

```

반환값

```

# 사람 있을때 - 여러명 예시
[
    {
        "person_id": 1,
        "bbox": (120, 50, 300, 480),
        "conf": 0.87,
        "class_name": "person"
    },
    {
        "person_id": 2,
        "bbox": (400, 80, 550, 500),
        "conf": 0.91,

```

```
        "class_name": "person"
    }
]

# 사람 없을 때
[]
```

클래스 흐름도

```
frame 입력
  ↓
YOLO 탐지
  ↓
person만 필터링
  ↓
ByteTrack이 ID 부여
  ↓
bbox 추출
  ↓
리스트 반환
```